CNN Lab Assignment

Task 2 — Convolutional Neural Networks

Name: Shashank
Roll Number: 24IT3056
Course: Machine Learning / Deep Learning
Framework: TensorFlow / Keras

In [1]:
```
import os, random
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import mnist, cifar10
from tensorflow.keras.utils import to_categorical

SEED = 42
random.seed(SEED); np.random.seed(SEED); tf.random.set_seed(SEED)
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '2'

gpus = tf.config.list_physical_devices('GPU')
if gpus:
    tf.config.experimental.set_memory_growth(gpus[0], True)
    print("GPU:", gpus[0].name)
else:
    print("no GPU")
```

```
GPU: /physical_device:GPU:0
```

In [2]:
```
def conv2d(image, kernel, stride=1, padding=0):
    if padding > 0:
        image = np.pad(image, padding, mode='constant')
    ih, iw = image.shape
    kh, kw = kernel.shape
    oh = (ih - kh) // stride + 1
    ow = (iw - kw) // stride + 1
    out = np.zeros((oh, ow))
    for i in range(oh):
        for j in range(ow):
            out[i,j] = np.sum(image[i*stride:i*stride+kh, j*stride:j*stride+kw] * k
    return out

img = np.array([[3,1,0,2,4],[1,5,3,2,1],[0,2,6,4,3],[2,3,1,5,2],[1,0,2,3,4]], dtype
sobel_x = np.array([[-1,0,1],[-2,0,2],[-1,0,1]], dtype=float)

res = conv2d(img, sobel_x, stride=1, padding=0)
print("Output shape:", res.shape)  # (3,3): (5-3+0)/1 + 1 = 3
print("Output:\n", res)
```

```
Output shape: (3, 3)
Output:
 [[ 7. -3. -3.]
 [13.  3. -7.]
 [ 5.  9.  1.]]
```

Problem 2 — Output Size Derivations

Formula: Output = floor((Input - Kernel + 2×Padding) / Stride) + 1

(a) Input=28, K=5, P=0, S=1 → floor((28-5)/1)+1 = **24×24**

(b) Input=28, K=3, P=1, S=1 → floor((28-3+2)/1)+1 = **28×28** (same padding)

(c) Input=32, K=3, P=0, S=2 → floor((32-3)/2)+1 = **15×15**

(d) First: K=3,P=1,S=1 on 32×32 → 32×32 | Second: K=3,P=0,S=1 → **30×30**

In [3]:
```
(x_tr, y_tr), (x_te, y_te) = mnist.load_data()
x_tr = np.expand_dims(x_tr.astype('float32')/255.0, -1)
x_te = np.expand_dims(x_te.astype('float32')/255.0, -1)
y_tr = to_categorical(y_tr, 10)
y_te = to_categorical(y_te, 10)

lenet = models.Sequential([
    layers.Conv2D(6,  (5,5), activation='tanh', input_shape=(28,28,1)),
    layers.AveragePooling2D((2,2)),
    layers.Conv2D(16, (5,5), activation='tanh'),
    layers.AveragePooling2D((2,2)),
    layers.Flatten(),
    layers.Dense(120, activation='tanh'),
    layers.Dense(84,  activation='tanh'),
    layers.Dense(10,  activation='softmax'),
], name='lenet5')

lenet.summary()
```

```
Model: "lenet5"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 conv2d (Conv2D)             (None, 24, 24, 6)         156

 average_pooling2d (AverageP  (None, 12, 12, 6)        0
 ooling2D)

 conv2d_1 (Conv2D)           (None, 8, 8, 16)          2416

 average_pooling2d_1 (Averag  (None, 4, 4, 16)         0
 ePooling2D)

 flatten (Flatten)           (None, 256)               0

 dense (Dense)               (None, 120)               30840

 dense_1 (Dense)             (None, 84)                10164

 dense_2 (Dense)             (None, 10)                850

=================================================================
Total params: 44,426
Trainable params: 44,426
Non-trainable params: 0
_________________________________________________________________
```

LeNet-5 Verification

Total params = **44,426** ✓ matches expected value

Manual count — first Conv2D:

Params = (K×K×C_in + 1) × C_out = (5×5×1 + 1) × 6 = 26 × 6 = **156** ✓

AvgPool vs MaxPool:

- LeNet used AvgPool (1998) — simpler, enough for MNIST
- MaxPool retains strongest feature per window, more robust to spatial shifts
- MaxPool gives better accuracy on complex datasets → modern default

In [4]:
```
(x_tr_c, y_tr_c), (x_te_c, y_te_c) = cifar10.load_data()
x_tr_c = x_tr_c.astype('float32')/255.0
x_te_c  = x_te_c.astype('float32')/255.0
y_tr_c  = to_categorical(y_tr_c.flatten(), 10)
y_te_c  = to_categorical(y_te_c.flatten(), 10)

custom = models.Sequential([
    layers.Conv2D(32,  (3,3), padding='same', input_shape=(32,32,3)),
    layers.BatchNormalization(), layers.ReLU(), layers.MaxPooling2D(),

    layers.Conv2D(64,  (3,3), padding='same'),
    layers.BatchNormalization(), layers.ReLU(), layers.MaxPooling2D(),
```

```
    layers.Conv2D(128, (3,3), padding='same'),
    layers.BatchNormalization(), layers.ReLU(), layers.MaxPooling2D(),

    layers.GlobalAveragePooling2D(),
    layers.Dense(256, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(10, activation='softmax'),
], name='custom_cnn')

custom.summary()
```

```
Model: "custom_cnn"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 conv2d_2 (Conv2D)           (None, 32, 32, 32)        896

 batch_normalization (BatchN  (None, 32, 32, 32)       128
 ormalization)

 re_lu (ReLU)                (None, 32, 32, 32)        0

 max_pooling2d (MaxPooling2D  (None, 16, 16, 32)       0
 )

 conv2d_3 (Conv2D)           (None, 16, 16, 64)        18496

 batch_normalization_1 (Batc  (None, 16, 16, 64)       256
 hNormalization)

 re_lu_1 (ReLU)              (None, 16, 16, 64)        0

 max_pooling2d_1 (MaxPooling  (None, 8, 8, 64)         0
 2D)

 conv2d_4 (Conv2D)           (None, 8, 8, 128)         73856

 batch_normalization_2 (Batc  (None, 8, 8, 128)        512
 hNormalization)

 re_lu_2 (ReLU)              (None, 8, 8, 128)         0

 max_pooling2d_2 (MaxPooling  (None, 4, 4, 128)        0
 2D)

 global_average_pooling2d (G  (None, 128)              0
 lobalAveragePooling2D)

 dense_3 (Dense)             (None, 256)               33024

 dropout (Dropout)           (None, 256)               0

 dense_4 (Dense)             (None, 10)                2570

=================================================================
Total params: 129,738
Trainable params: 129,290
Non-trainable params: 448
_________________________________________________________________
```

Custom CNN — Architecture & Rationale

```
Input (32×32×3)
→ Conv2D(32) + BN + ReLU + MaxPool  →  16×16×32
→ Conv2D(64) + BN + ReLU + MaxPool  →  8×8×64
→ Conv2D(128)+ BN + ReLU + MaxPool  →  4×4×128
```

```
→ GlobalAveragePooling              →  128
→ Dense(256) + Dropout(0.5)
→ Dense(10, softmax)
```

- filters double each block (32→64→128): low-to-high level features
- BatchNorm after each conv: stabilises and speeds training
- GAP instead of Flatten: keeps params low, translation-invariant
- Dropout(0.5) in head: prevents classifier overfitting
- total params ~129K — within 200K–2M target

Analysis Questions

Q1.

- two 3×3 (F=64): 2×(9×64+1)×64 = 73,856 params
- one 5×5 (F=64): (25×64+1)×64 = 102,464 params
- two 3×3 use ~28% fewer params, same 5×5 receptive field, extra non-linearity

Q2.

- BatchNorm normalises each mini-batch to zero mean, unit variance; learns γ,β
- placed after Conv, before activation
- benefits: higher LR allowed, training more stable, acts as mild regularisation

Q3.

- GAP averages H×W per channel → flat C-vector; translation-invariant
- Flatten on 4×4×128: 2048 inputs to Dense(256) → 524K params vs 33K with GAP
- GAP = fewer params, less overfitting, better generalisation